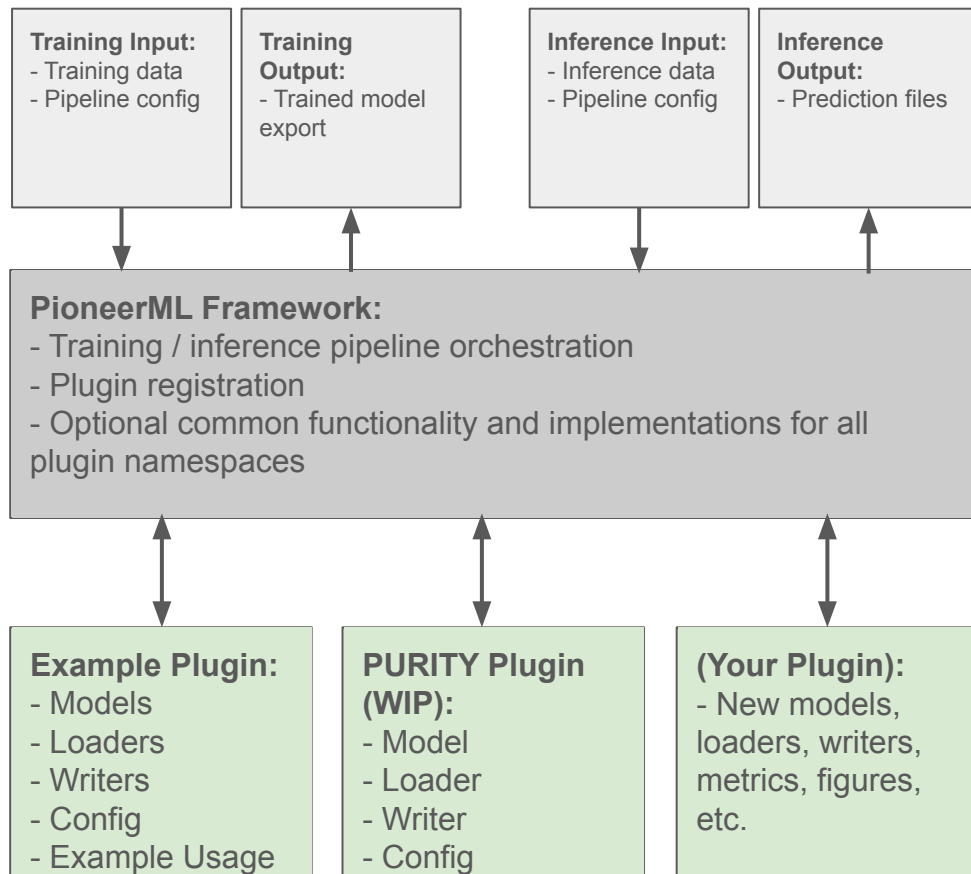


State of ML Reconstruction Framework

Jack Carlton
University of Kentucky

Plugin Based Architecture

- Design goals:
 - Enable modular extensions for:
 - Models
 - Loaders
 - Writers
 - Etc.
 - Abstract common functionality:
 - Training / inference pipeline orchestration
 - Data streaming
 - Model I/O (export / import)
 - Etc.
- Approach
 - Plugin registry system with configuration-driven abstraction
 - Leverages reflection for configuration
 - Plugins may depend on each other



Existing Plugin Namespaces

- **Architecture** - trainable models
- **Module** - lightning modules (handle training steps)
 - **Loss** - loss functions used by the module during training
- **Loader Manager** - manages multiple loaders (e.g. train, validation, test loaders)
 - **Loader** - take input files and create training/inference batches for hardware
- **Writer** - take model predictions and form output files
- **Trainer** - orchestrates model training, early stopping, etc.
- **Exporter** - defines trained model export strategy/format
- **Model Handle** - defines model import and loading strategy
- **Evaluator** - defines how validation is calculated, as well as any auxiliary metrics
 - **Metric** - objects to evaluate a model (e.g. percent of false positives)
 - **Plot** - figures to evaluate a model (e.g. loss curves)
- **Hyper Parameter Optimization (HPO)** - orchestrates hpo search config
 - **Objective** - What to optimize for in the hpo search
 - **Search space** - what parameters to vary (and how to vary)

Example: Register Plugin Component

- Registering components allows them to be instantiated from config
- Models defined via config → swappable without changing pipeline code
 - Can test new configurations without writing new code (only new config)
- To be registered the class must derive from some base
 - In this example `BaseGraphClassifierModel` derives from `BaseArchitecture`, which is registerable

```
"""group classifier."""

from __future__ import annotations

from
pioneerml.integration.pytorch.models.architectures.graph.t
ransformer.classifiers.base_graph_classifier_model import
BaseGraphClassifierModel
from
pioneerml.integration.pytorch.models.architectures.factory
.registry import REGISTRY as ARCHITECTURE_REGISTRY

#register name + aliases
@ARCHITECTURE_REGISTRY.register("group_classifier_stereo")
@ARCHITECTURE_REGISTRY.register("group_classifier")
class GroupClassifierStereo(BaseGraphClassifierModel):
    # define parameters (deduced later by abstraction)
    def __init__(
        self,
        node_dim: int = 4,
        edge_dim: int = 4,
        graph_dim: int = 0,
        hidden: int = 200,
        heads: int = 4,
        num_blocks: int = 2,
        dropout: float = 0.1,
        num_classes: int = 3,
    ):
    ...
```

Example: Creating Plugin Component from Config

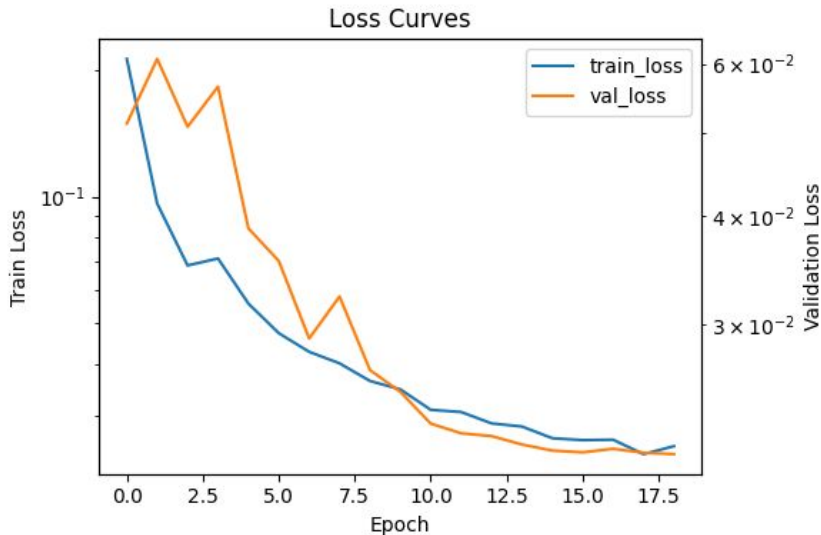
- Registered plugin components can be represented as config and dynamically built by factories
- Pipeline steps may required component types
 - e.g. the training training step requires an architecture (model) component

```
"architecture": {  
  "type": "group_classifier",  
  "config": {  
    "node_dim": 4,  
    "edge_dim": 4,  
    "graph_dim": 0,  
    "hidden": 128,  
    "heads": 4,  
    "num_blocks": 2,  
    "dropout": 0.1  
  }  
},
```

Example config representation of a group classifier model

Example: Training Model

- Many components can be composed in config to create a training pipeline
 - E.g. the hpo expect components for:
 - Architecture
 - Module
 - Trainer
 - Loader Manager
 - Loaders
 - HPO
 - Objective
 - Search space
- [Example plugin](#), [example config](#), and [example notebook](#) available (still WIP)



Example loss curve generated from training a group classifier model

Steps Before First ML Results with Simulation

- Develop the “unified model” (Omar) + develop pioneerML framework to handle scaled training + inference (Jack)
 - Framework in a functioning state but needs cleanup for integration of new models/inference strategies
- Integrate the unified model into the pioneerML repository
- Integrate ML ready dataset generator sub algorithm into the new v3 simulation
- Train the unified model on a dataset generated from the simulation framework
- Decide how inference will be done with the trained model
 - My suggestion:
 - Add pioneerML repo to the simulation framework
 - Add trained model to the simulation framework
 - Write a thin python Gaudi wrapper around the inference pipeline provided
 - This pipeline can be designed to run a Triton based inference to a server hosting a set of GPUs
- Run full simulation, compare ML inferences with traditional reco directly, etc.

Auxiliary Slides